



Código Siempre Verde

Testing y Arquitectura Limpia en Spring Boot

La Filosofía de la Calidad



Las 3 Dimensiones de Calidad

Correctness

¿El código hace lo que debe?

Regression Safety

¿Un cambio nuevo rompe algo existente?

Documentation

Un test bien escrito es el contrato de la unidad.

Anatomía de un Test Unitario

Arrange (Preparar)

Configurar datos,
inyectar dependencias
y mocks.

Act (Ejecutar)

Llamar al método
específico (¡Solo una
acción lógica por test!).

Assert (Verificar)

Validar los resultados
y el estado final.

Regla F.I.R.S.T.
Fast, Independent, Repeatable, Self-validating, Timely.

⚠ ALERTA: Evitar `new Date()` o valores aleatorios.
Rompen la repetibilidad.

El Ecosistema JUnit 5

@BeforeEach

Setup. Crea el aislamiento reiniciando el estado antes de cada prueba.

@Test

Define el contrato. El ejecutor principal.

@AfterEach

Teardown. Limpia recursos (crucial para evitar contaminación entre pruebas).

Assertions Clave

assertEquals

Para validar el camino feliz (happy path).

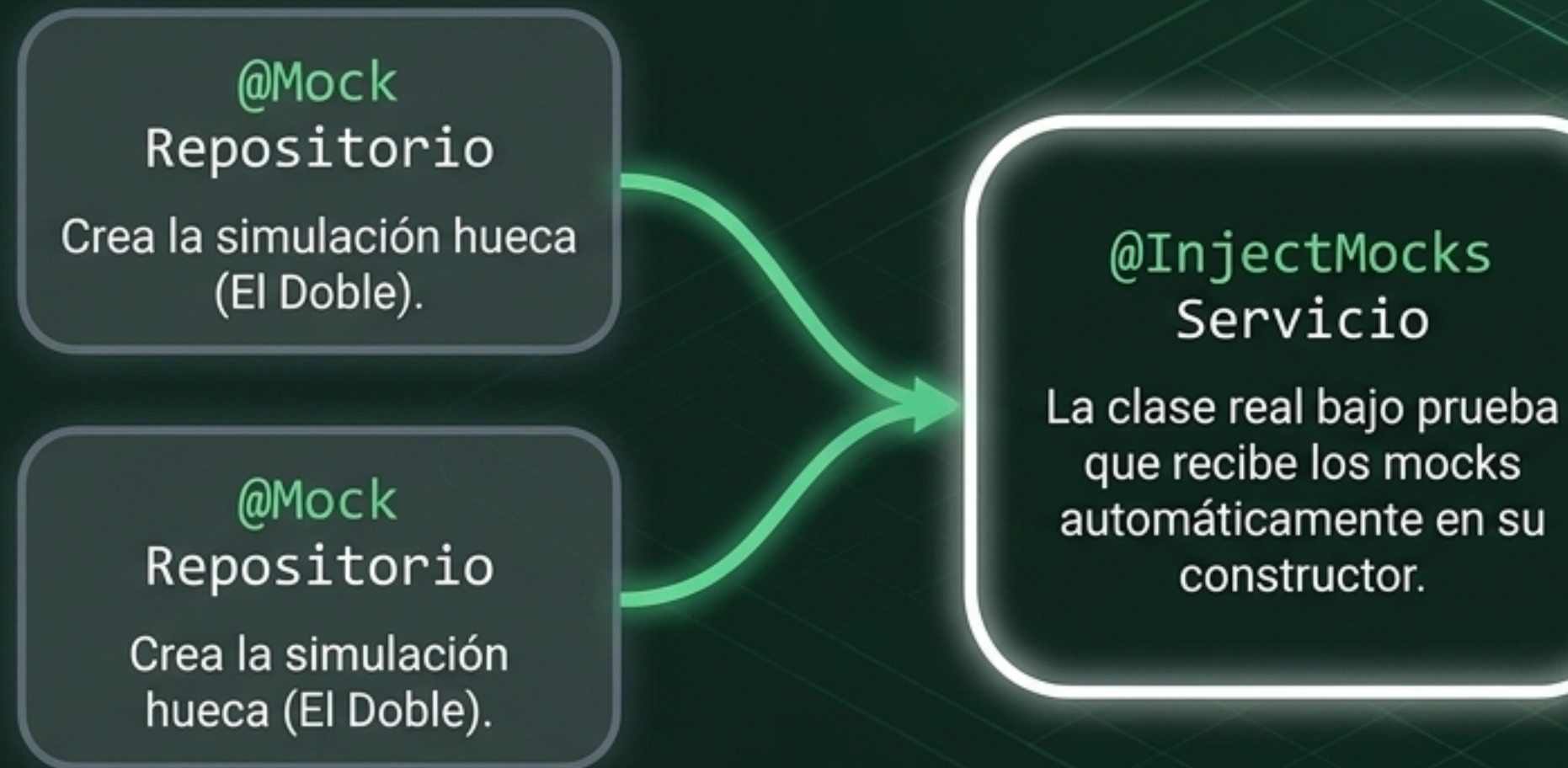
assertThrows

Para validar el manejo seguro de excepciones.

El Arsenal de Mockito

	Tipo	Comportamiento	Uso Principal
	Dummy	No	Solo relleno de parámetros requeridos.
	Fake	Sí	Implementación simple pero funcional (ej. In-memory DB).
	Stub	Parcial	Responde estáticamente con valores pre-programados.
	Spy	Real	Envuelve un objeto real y registra secretamente sus interacciones.
	Mock	Simulado	Expectativas configuradas + verificación estricta de interacciones.

Mockito en Acción



Control Panel

Controlar

```
when(mock.metodo()).thenReturn(valor)
```

Domina el universo simulado.

Auditar

```
verify(mock).metodo()
```

Comprueba los efectos colaterales sin ejecutar el código real.

Aislamiento Quirúrgico con Test Slicing

El Problema:

Usar `@SpringBootTest` arranca todo (Tomcat, DB, todos los beans). Para suites grandes, esto significa minutos de espera.

La Solución:

Levantar solo la rebanada (slice) del contexto de Spring que necesitas probar.

El Impacto:

Velocidad exponencial y aislamiento total de fallos.

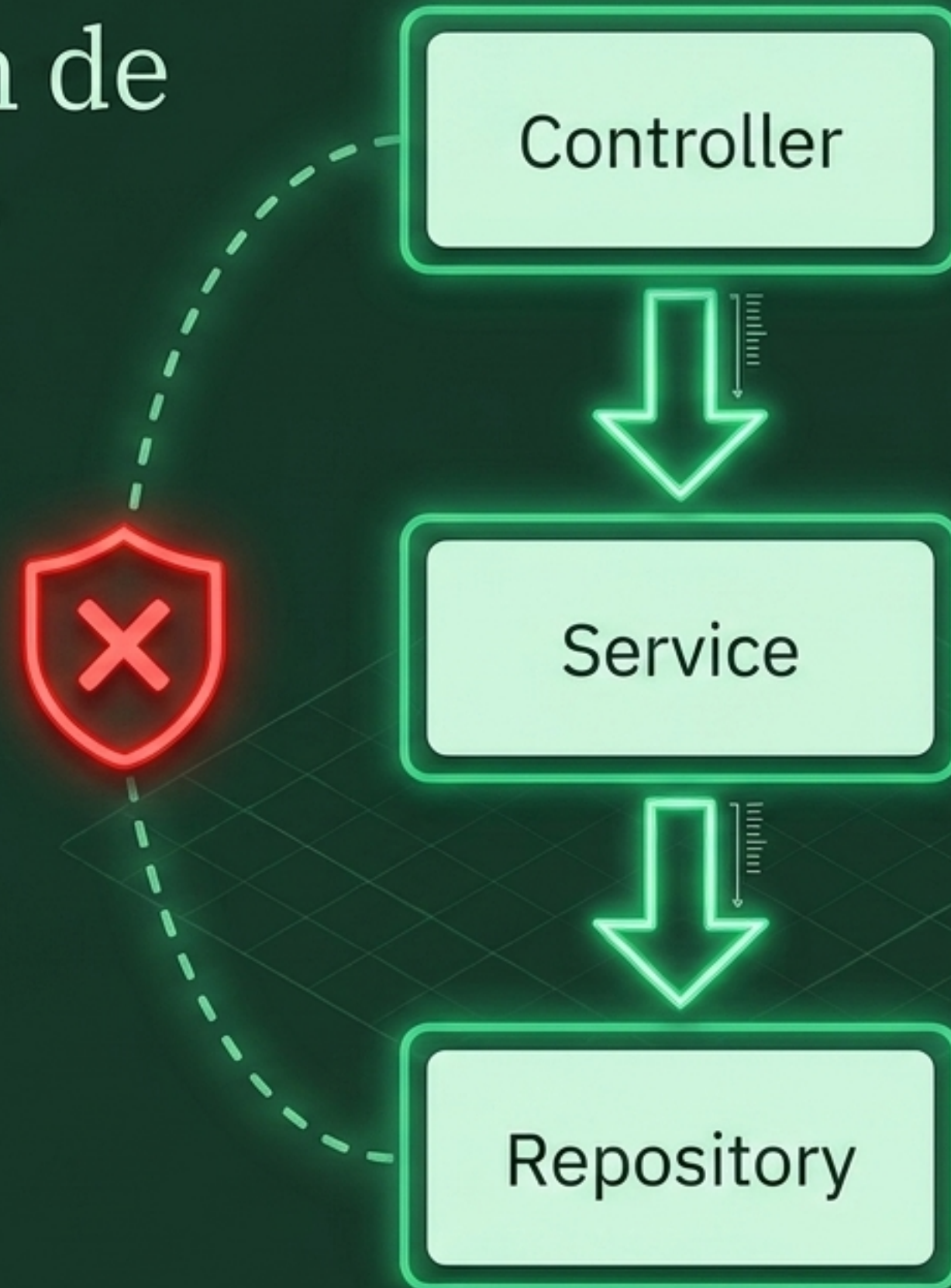


⚠ PRECAUCIÓN (Caché): Spring cachea los contextos. No varíes tus mocks innecesariamente entre pruebas del mismo slice o destruirás el rendimiento de la caché.

¿Qué anotación de Spring usar?

Anotación Tag	Velocidad	Descripción
@JsonTest	 Muy Rápida	Prueba exclusiva de serialización Jackson.
@WebMvcTest	 Rápida	Prueba Controllers + MockMvc . Requiere simular Servicios con @MockBean .
@DataJpaTest	 Media	Prueba Repositorios + BD (H2) . Ventaja: Ejecuta un rollback automático al finalizar.
@SpringBootTest	 Lenta	Carga todo el contexto. Reservado exclusivamente para flujos End-to-End.

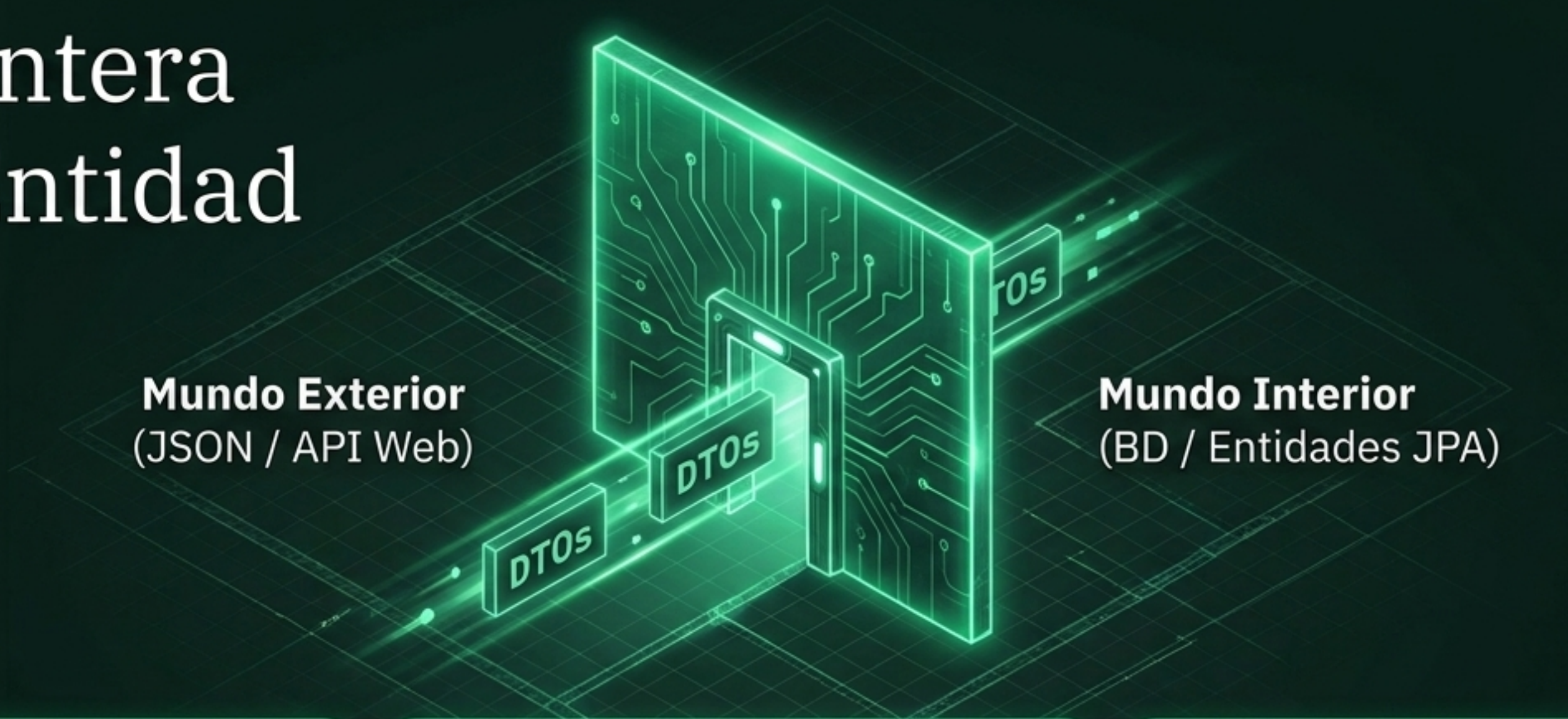
El Guardián de las Pruebas



Arquitectura de Capas

- La regla de oro inquebrantable: Las capas no se saltan.
- La dependencia siempre fluye en una sola dirección hacia abajo.
- Por qué importa: Si el Controller importa directamente el Repository, la separación de preocupaciones colapsa y el `@WebMvcTest` se vuelve imposible de aislar.

La Frontera de la Entidad



El Riesgo

Exponer entidades JPA directamente filtra la estructura de la base de datos y abre vulnerabilidades de inyección (Mass Assignment).

La Defensa

Objetos de Transferencia de Datos (DTOs). El único vehículo permitido para cruzar la frontera arquitectónica.

El Estándar Moderno

Utilizar Records de Java (16+) para garantizar que estos objetos sean inmutables por diseño.

Principios SOLID en Spring (I)

	Síntoma	Solución Spring
	S - Single Responsibility Clases Service de +1000 líneas.	Dividir megaservicios en componentes <u>granulares y ultra-específicos</u> .
	O - Open/Closed Modificar lógica existente rompe tests pasados.	Injectar List<Interface> y agregar nuevos Beans en lugar de modificar bloques switch.
	L - Liskov Substitution Uso excesivo de validaciones instanceof y casteos forzados.	Si un servicio depende de Repository, debe funcionar idénticamente con MySQL en prod y H2 en test.

Principios SOLID en Spring (II)

...	Síntoma	Solución Spring
 I - Interface Segregation	Clases implementando métodos inútiles que lanzan <code>NotImplementedException</code> .	Crear múltiples interfaces pequeñas de servicio en lugar de una interfaz "Dios".
...	Síntoma	Solución Spring
 D - Dependency Inversion	Usar explícitamente <code>new ClaseConcreta()</code> en medio de la lógica de negocio.	El corazón de Spring Boot. Los servicios dependen de abstracciones (interfaces), y el framework inyecta la implementación por constructor.

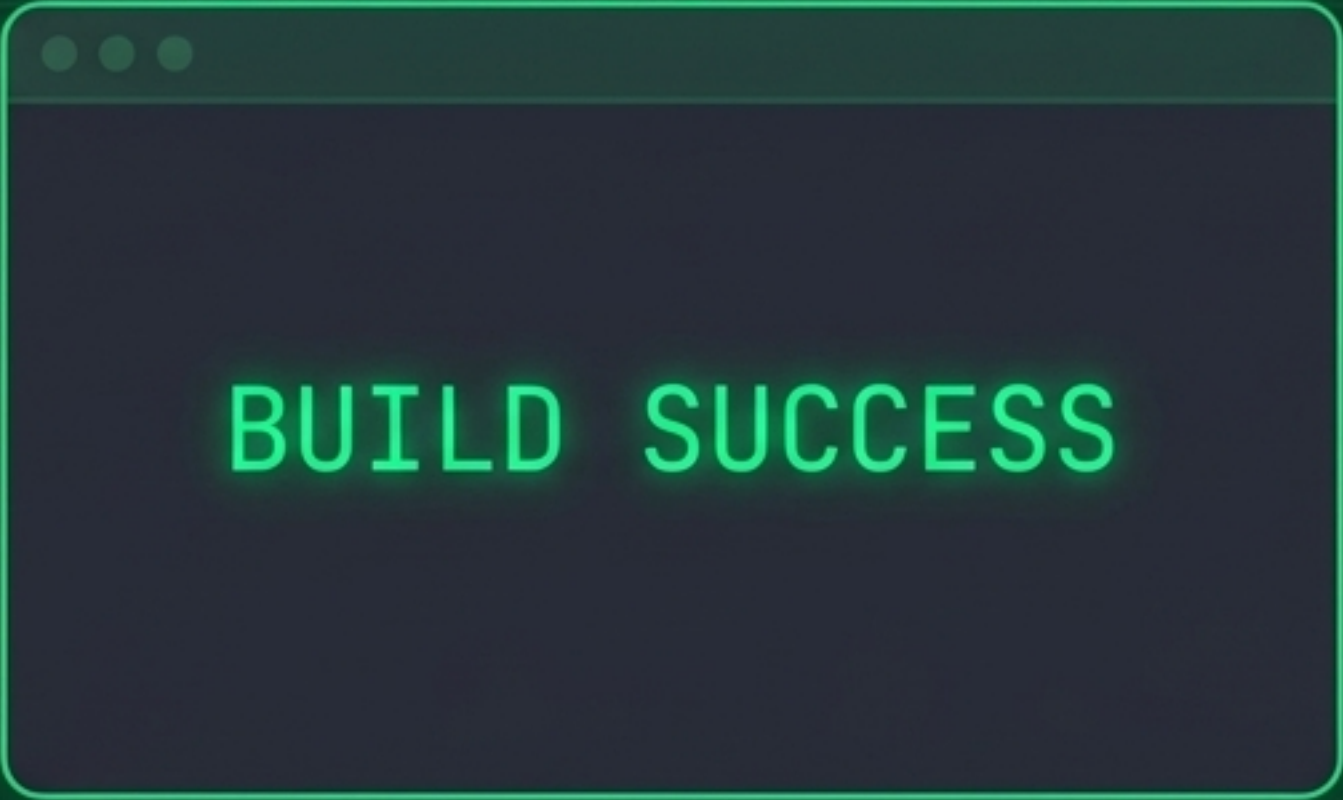
La Simbiosis



Conclusión: Arquitectura Limpia es lo que habilita el Test Slicing. Si no puedes probar un componente en aislamiento, el problema no es tu herramienta de test, ES TU ARQUITECTURA.

El Estándar Green Build

- [✓] Dependencias inyectadas exclusivamente por constructor (garantiza inmutabilidad).
- [✓] Entidades JPA confinadas; nunca tocan la capa web (uso estricto de DTOs).
- [✓] Tests sin lógica condicional (if/for), respetando estrictamente el patrón Arrange-Act-Assert.
- [✓] Test Slices aplicados correctamente para evitar levantar un servidor Tomcat innecesario.
- [✓] Convenciones de nomenclatura respetadas (el nombre del test actúa como documentación viva).



BUILD SUCCESS

**“El código no es legado porque sea viejo;
es legado porque no tiene pruebas.”**

Una arquitectura sólida y una suite de pruebas granulares son el único camino hacia aplicaciones verdaderamente escalables, seguras y Siempre Verdes.

Próximo paso: Integración total en el Proyecto Final.